

مستند معماری هوش مصنوعی AI Chat SaaS

AI Architecture Documentation

معماری دستیار هوشمند دانش محور برای پیشنهاد پاسخ، پاسخ خودکار، خزیدن محتوای سایت، بازیابی دانش و چرخه بهبود پرسش های بی پاسخ

نام پروژه:	AI Chat SaaS
نوع سند:	مستند معماری هوش مصنوعی
نسخه سند:	1.0
تاریخ:	۱۴ جولای ۲۰۲۶
نویسنده/تهیه کننده:	مبین یوسفی
مسیر سند:	doc/LaTeX/05-AI-Architecture/05-AI-Architecture.tex

فهرست مطالب

۳	۱ هدف و دامنه سند
۳	۲ نمای کلی معماری AI
۴	۳ اجزای پیاده سازی
۶	۴ Pipeline اصلی AI
۶	۱.۴ Pipeline ساخت دانش
۶	۲.۴ Pipeline پاسخ دهی
۷	۵ Pipeline NLP
۸	۱.۵ نرمال سازی متن
۸	۲.۵ توکن سازی سوال
۸	۳.۵ تشخیص نیت و دسته بندی
۸	۶ Engineering Prompt و منطق قالب ها
۸	۱.۶ قالب تولید سوال
۹	۲.۶ قالب پاسخ استخراجی
۹	۷ Database Vector و Embedding
۹	۸ Retrieval Document و Search Semantic
۱۰	۱.۸ Ranking
۱۰	۹ RAG در معماری فعلی
۱۱	۱۰ Extraction Entity
۱۱	۱۱ Management Context و Memory
۱۱	۱۲ Generation Response
۱۲	۱۳ Logic Fallback
۱۲	۱۴ Caching
۱۳	۱۵ Selection Model

۱۳	Reasoning ۱۶
۱۳	Tuning Fine ۱۷
۱۴	Evaluation ۱۸
۱۴	کنترل‌های امنیتی و دسترسی در AI ۱۹
۱۵	محدودیت‌های فعلی و مسیر توسعه ۲۰
۱۵	جمع‌بندی ۲۱

۱ هدف و دامنه سند

هدف این سند توضیح دقیق این است که لایه هوش مصنوعی پروژه AI Chat SaaS چگونه کار می‌کند، داده را از چه منابعی می‌گیرد، چگونه دانش سایت مشتری را آماده می‌کند، چگونه سوال کاربر را تحلیل می‌کند، چگونه منبع مناسب را پیدا می‌کند، چگونه پاسخ می‌سازد و در چه شرایطی پاسخ مستقیم نمی‌دهد.

این سند به صورت مشخص موارد خواسته شده در Rahnama.md برای «مستند معماری هوش مصنوعی» را پوشش می‌دهد:

- NLP Pipeline و Pipeline
- Prompt Engineering و منطق قالب‌های پرسش
- Vector Database، Embedding و وضعیت فعلی آن‌ها
- Semantic Search، بازیابی مستندات و امتیازدهی
- Entity Extraction و Intent Detection
- RAG و Ranking، Document Retrieval
- Context Management و Memory
- Caching و Fallback Logic، Response Generation
- Evaluation و Fine Tuning، Reasoning، Model Selection

نکته مهم معماری این است که موتور فعال فعلی در مسیرهای اصلی محصول، یک سیستم دانش‌محور، امتیازدهی‌شده و استخراجی است. یعنی برای هر پاسخ الزاما به مدل زبانی خارجی متکی نیست؛ بلکه از دانش دستی، محتوای خزیده شده سایت، سوال‌های تولیدشده از محتوا، تشخیص نیت، جست‌وجوی متنی، تقویت امتیاز و آستانه اطمینان استفاده می‌کند. متغیرهای OPENAI_API_KEY و OPENAI_MODEL در پیکربندی وجود دارند، اما جریان فعال پاسخ‌دهی در فایل backend/includes/ai-answer-engine.php بر پایه جست‌وجوی محلی و ساخت پاسخ استخراجی پیاده‌سازی شده است.

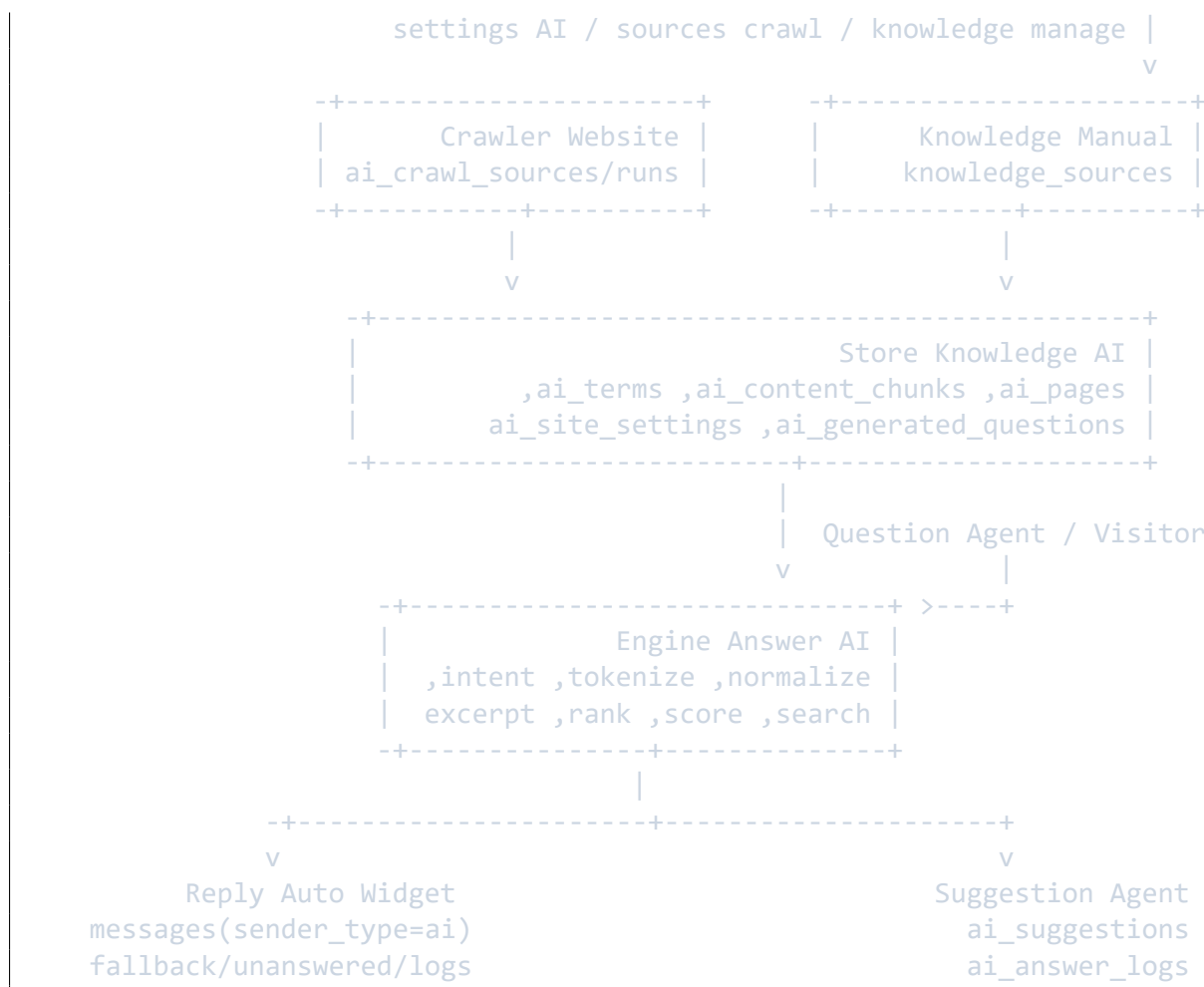
۲ نمای کلی معماری AI

لایه هوش مصنوعی پروژه از سه زیرسیستم اصلی تشکیل شده است:

۱. **زیرسیستم ساخت دانش:** شامل دانش دستی، منابع خزیدن سایت، خزنده، استخراج متن خوانا، قطعه‌بندی محتوا، استخراج واژه‌های مهم و تولید سوال‌های احتمالی.
۲. **زیرسیستم بازیابی و رتبه‌بندی:** شامل نرمال‌سازی سوال، استخراج توکن‌ها، تشخیص نیت، جست‌وجو در منابع دانش، امتیازدهی، تقویت با termها و انتخاب بهترین نامزد.
۳. **زیرسیستم مصرف پاسخ:** شامل پیشنهاد پاسخ برای agent، پاسخ خودکار widget در زمان آفلاین بودن پشتیبانی، fallback ثبت لاگ و ثبت سوال‌های بی‌پاسخ برای بهبود آینده.

نمودار ۱: معماری کلان لایه AI

Customer Admin



۳ اجزای پیاده‌سازی

فایل/جدول	نقش در معماری AI
ai-crawler.php	خزیدن صفحات، استخراج عنوان، توضیحات متا، تیتراژ اصلی، متن خوانا، لینک‌ها، قطعه‌بندی متن، استخراج term و تولید سوال‌های قالبی.
ai-answer-engine.php	هسته پاسخ‌دهی؛ نرمال‌سازی سوال، استخراج توکن، تشخیص نیت، جست‌وجو در دانش دستی، chunks و ques-generated، امتیازدهی و ساخت پاسخ استخراجی.
ai-helpers.php	توابع کمکی AI شامل تبدیل مقادیرها، محدودسازی score، دسترسی سایت مشتری، نرمال‌سازی host و اعتبارسنجی منبع crawl.
ai-crawl-start.php	شروع crawl برای سایت مشتری، خواندن تنظیمات، صف URLها، اجرای خزیدن همزمان و ثبت summary اجرای crawl.
ai-suggestion-generate.php	تولید پیشنهاد پاسخ برای agent با کنترل نقش، دسترسی سایت، قابلیت plan و آستانه min_suggestion_score.

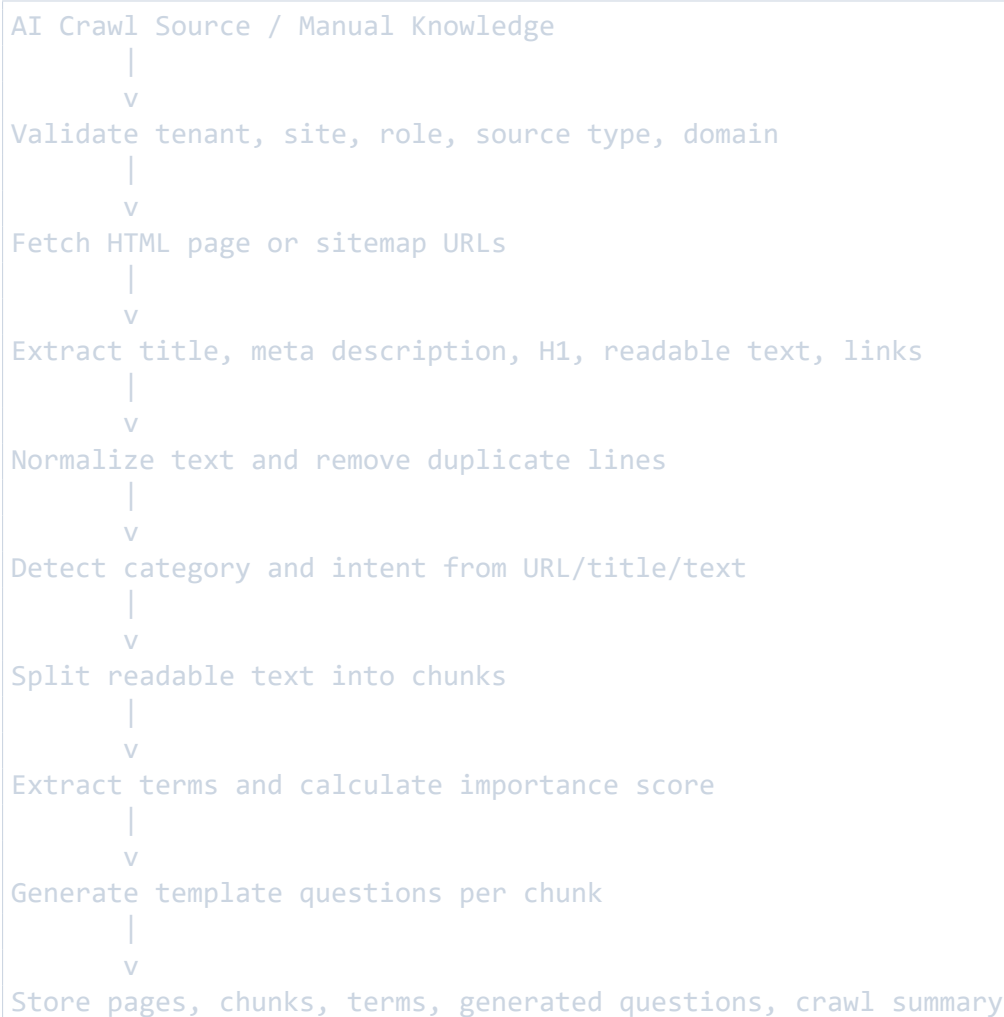
ai-reply.php	پاسخ خودکار به visitor در widget فقط در صورت فعال بودن AI، آفلاین بودن پشتیبانی و عبور امتیاز از min_auto_reply_score.
ai_site_settings	تنظیمات AI در سطح سایت؛ فعال/غیرفعال بودن، assistant، crawl، reply، auto fall-back، حد اکثر صفحات و پیام.
ai_pages	صفحات خزیده شده همراه با URL، عنوان، تیتر، متن تمیز، دسته بندی، intent، وضعیت crawl و hash محتوا.
ai_content_chunks	قطعه های قابل جست و جو از صفحات، همراه با متن نرمال شده، score، importance و keywords intent، category،
ai_terms	های term استخراج شده از هر chunk برای تقویت امتیاز تطبیق.
ai_generated_questions	سوال های احتمالی تولید شده با template از محتوای هر chunk.
ai_answer_logs	لاگ همه تلاش های پاسخ دهی، score، متن پاسخ، منابع های استفاده شده و mode پاسخ.
ai_unanswered_questions	صف سوال هایی که پاسخ مطمئن نگرفته اند و باید توسط مشتری بررسی و به دانش اضافه شوند.

۴ Pipeline اصلی AI

جریان AI در محصول دو ورودی اصلی دارد: ساخت دانش و پاسخ‌دهی. ساخت دانش قبل از پاسخ‌دهی انجام می‌شود و خروجی آن پایگاه دانشی است که موتور پاسخ روی آن کار می‌کند.

۱.۴ Pipeline ساخت دانش

نمودار ۲: Pipeline Ingestion Knowledge



در این pipeline، منابع crawl در جدول `ai_crawl_sources` تعریف می‌شوند و می‌توانند از نوع `url`، `path_prefix` یا `sitemap` باشند. `endpoint` شروع crawl، منابع فعال را می‌خواند، تنظیمات سایت را اعمال می‌کند، URLها را در صف قرار می‌دهد و تا سقف `max_pages_per_crawl` و `max_depth` صفحات را دریافت می‌کند. هر URL باید به دامنه همان سایت تعلق داشته باشد؛ این کنترل با توابع `ai_validate_crawl_source` و `ai_host_belongs_to_site` انجام می‌شود.

۲.۴ Pipeline پاسخ‌دهی

نمودار ۳: Pipeline Answer

```

Visitor or Agent Question
|
v
Normalize question
|
v
Extract meaningful tokens and remove stop words
|
v
Detect category and intent
|
v
Search candidates:
- knowledge_sources
- ai_generated_questions
- ai_content_chunks
|
v
Score candidates:
token match + question match + answer match
+ intent/category boost + term boost
+ manual knowledge boost + FAQ/approved boost
|
v
Rank candidates and select best answer
|
v
Build extractive reply from best source text
|
v
Apply threshold:
suggestion >= min_suggestion_score
auto reply >= min_auto_reply_score
|
v
Return answer or fallback, log result, save unanswered if needed

```

تابع مرکزی این جریان `ai_find_best_answer()` است. این تابع خروجی‌هایی مانند `matched_question_id`، `matched_chunk_id`، `sources`، `answer`، `confidence_score` و `best_candidates` تولید می‌کند. سپس `endpoint` مصرف‌کننده تصمیم می‌گیرد آیا پاسخ به `agent` پیشنهاد شود، مستقیم در `widget` ارسال شود یا `fallback` فعال شود.

۵ Pipeline NLP

پردازش زبان طبیعی در نسخه فعلی سبک و `rule-based` است. هدف آن ساخت یک سیستم قابل توضیح، قابل کنترل و وابسته به دانش تاییدشده مشتری است.

۱.۵ نرمال سازی متن

تابع `ai_normalize_text()` کارهای زیر را انجام می دهد:

- تبدیل HTML ها entity به متن قابل خواندن.
- یکسان سازی برخی نویسه های عربی و فارسی مانند «ي» به «ی» و «ك» به «ک».
- حذف فاصله های تکراری و مرتب سازی شکست خطها.
- آماده سازی متن برای ذخیره، جست و جو و امتیازدهی.

۲.۵ توکن سازی سوال

تابع `ai_question_tokens()` سوال را کوچک سازی، پاک سازی و به توکن تبدیل می کند. سپس کلمات توقف فارسی و انگلیسی حذف می شوند و حداکثر ۱۲ توکن متمایز برای جست و جو نگه داشته می شود. این محدودیت باعث می شود های SQL query کنترل شده تر بمانند و سوال های طولانی، موتور جست و جو را بیش از حد گسترده نکنند.

۳.۵ تشخیص نیت و دسته بندی

تابع `ai_detect_question_intent()` با keyword ها intent rule سوال را تشخیص می دهد. in-های tent فعلی شامل موارد زیر هستند:

- pricing: قیمت، هزینه، تعرفه و پرداخت
- contact: تماس، آدرس، شماره تماس و موقعیت
- appointment: نوبت دهی، رزرو و وقت مشاوره
- shipping: ارسال، تحویل و مرسوله
- faq: سوالات متداول
- service_info: اطلاعات خدمات یا محصول
- general_info: حالت عمومی در صورت نبود تطبیق مشخص

۶ Engineering Prompt و منطق قالبها

در جریان فعال فعلی، prompt به معنای ارسال متن به LLM خارجی استفاده نمی شود. اما سیستم از «قالب های تولید سوال» و «قالب پاسخ استخراجی» استفاده می کند که نقش مشابهی در کنترل خروجی دارند.

۱.۶ قالب تولید سوال

تابع `ai_generate_questions_from_chunk()` براساس intent هر chunk سوال های احتمالی تولید می کند. برای مثال اگر intent برابر pricing باشد، سوال هایی درباره هزینه و تعرفه ساخته می شود؛ اگر intent برابر appointment باشد، سوال هایی درباره رزرو و نوبت ساخته می شود. این

سوال‌های قالبی در `ai_generated_questions` ذخیره می‌شوند و هنگام پاسخ‌دهی به‌عنوان نامزد بازیابی استفاده می‌شوند.

۲.۶ قالب پاسخ استخراجی

تابع `ai_build_excerpt_reply()` متن منبع منتخب را به جمله‌های کوتاه‌تر تقسیم می‌کند، جمله‌های نزدیک‌تر به سوال را امتیازدهی می‌کند و نهایتاً یک پاسخ کوتاه با پیشوند «طبق اطلاعات سایت» می‌سازد. این طراحی باعث می‌شود پاسخ به منبع واقعی سایت متکی باشد و از تولید آزاد و غیرقابل کنترل جلوگیری شود.

۷ Database Vector و Embedding

در نسخه فعلی، پروژه جدول `embedding` یا پایگاه برداری اختصاصی ندارد. جست‌وجوی فعال با `LIKE`، `FULLTEXT` های `MySQL`، توکن‌های نرمال‌شده، `boost term` و امتیازدهی `rule-based` انجام می‌شود. بنابراین وضعیت فعلی معماری به شکل زیر است:

مفهوم	وضعیت در نسخه فعلی
Embedding	پیاده‌سازی نشده است. معادل عملی فعلی، نرمال‌سازی متن، استخراج <code>term</code> و <code>scoring</code> متنی است.
Vector Database	وجود ندارد. پایگاه اصلی <code>MySQL</code> است و جداول <code>AI</code> با <code>index</code> و <code>FULLTEXT</code> آماده شده‌اند.
Semantic Search	به‌صورت کامل برداری نیست؛ به‌صورت جست‌وجوی معنایی سبک با <code>boost</code> ، <code>term intent/category</code> ، سوال‌های تولیدی و رتبه‌بندی ترکیبی پیاده‌سازی شده است.
Future Path	امکان افزودن ستون‌های <code>embedding</code> برای <code>chunks</code> و سوال‌ها، و اتصال به <code>Milvus Qdrant</code> ، <code>pgvector</code> یا سرویس مشابه در نسخه‌های بعدی وجود دارد.

این تصمیم برای MVP منطقی است، چون پاسخ‌ها قابل توضیح و قابل کنترل هستند، هزینه `inference` خارجی کاهش می‌یابد و محصول می‌تواند بدون وابستگی کامل به سرویس ثالث کار کند. در عین حال برای دقت معنایی بهتر در سوال‌های پارافرایز شده، افزودن `embedding` در مسیر توسعه آینده توصیه می‌شود.

۸ Retrieval Document و Search Semantic

موتور جست‌وجوی پاسخ سه منبع را همزمان بررسی می‌کند:

۱. دانش دستی: جدول `knowledge_sources` که توسط `admin customer` ایجاد می‌شود.

۲. سوال‌های تولیدشده: جدول `ai_generated_questions` که از `chunks` ساخته شده است.

۳. **قطعه‌های محتوایی:** جدول `ai_content_chunks` که متن صفحات خزیده‌شده را نگه می‌دارد.

برای هر منبع، های `candidate` مرتبط با توکن‌های سوال بازیابی می‌شوند. سپس موتور به هر `candidate` امتیاز می‌دهد. دانش دستی به صورت عمدی `boost` بالاتری دریافت می‌کند، چون داده‌ای است که مشتری آگاهانه وارد کرده و معمولاً دقیق‌تر از متن خام سایت است.

۱.۸ Ranking

امتیاز نهایی از ترکیب چند عامل ساخته می‌شود:

- تعداد و وزن توکن‌های تطبیق‌یافته.
- تطبیق سوال اصلی با عنوان، سوال یا متن منبع.
- تطبیق `intent` و `category` تشخیص‌داده‌شده.
- `boost term` از جدول `ai_terms`.
- `score importance` هر `chunk`.
- اولویت دانش دستی، وضعیت `approved` و نوع `faq`.

خروجی `ranking` فقط یک `answer` نیست؛ بلکه `sources top` و `candidates best` را هم برمی‌گرداند تا در `analytics debug` و بهبود آینده قابل استفاده باشند.

۹ RAG در معماری فعلی

اگر RAG را به معنای «بازیابی دانش مرتبط و ساخت پاسخ بر پایه آن دانش» در نظر بگیریم، معماری فعلی یک RAG استخراجی و `rule-based` است. تفاوت آن با RAG متداول محور LLM این است که مرحله `generation` با LLM انجام نمی‌شود؛ بلکه پاسخ از متن منبع منتخب استخراج و خلاصه‌سازی سبک می‌شود.

نمودار ۴: RAG استخراجی فعلی

```

Question
|
v
Retriever:
  manual knowledge + generated questions + chunks
  |
  v
Ranker:
  token score + intent/category + term boost
  |
  v
Context:
  best source text + top sources
  |
  v
Generator:

```

```

extract best sentences from source text
|
v
Answer / Fallback / Unanswered Queue

```

این روش برای پشتیبانی مشتری مزیت دارد، چون پاسخ‌ها به محتوای تاییدشده سایت محدود می‌شوند و ریسک پاسخ‌سازی بی‌پایه کاهش می‌یابد.

۱۰ Extraction Entity

در نسخه فعلی extraction entity به صورت یک مازول مستقل پیاده‌سازی نشده است. اما برخی نشانه‌های معنایی از متن استخراج می‌شوند:

- های term پرتکرار و مهم از هر chunk در ai_terms.
- category و intent برای chunk page، question.
- URL، description meta title و heading main به عنوان metadata قابل بازیابی.

برای نسخه‌های بعدی، هایی entity مانند نام محصول، خدمت، شهر، شماره تماس، قیمت، تاریخ، ساعت کاری و نام شعبه می‌توانند به صورت ساختاریافته استخراج و در جداول جداگانه یا metadata JSON ذخیره شوند.

۱۱ Management Context و Memory

معماری فعلی دو نوع حافظه دارد:

۱. **حافظه مکالمه:** پیام‌ها در messages و گفتگوها در conversations ذخیره می‌شوند. suggestion AI از آخرین پیام visitor در conversation استفاده می‌کند.
۲. **حافظه دانشی:** دانش دستی، صفحات خزیده‌شده، terms chunks و ques-generated در جداول AI ذخیره می‌شوند و بین مکالمات قابل استفاده‌اند.

در نسخه فعلی، پاسخ‌دهی AI تاریخچه کامل مکالمه را به عنوان context چندمرحله‌ای تحلیل نمی‌کند. تمرکز آن روی آخرین سوال visitor و دانش سایت است. این انتخاب ساده و قابل کنترل است، اما برای سوال‌های پیگیری مانند «قیمت همان مورد چقدر است؟» نیاز به توسعه resolver context وجود دارد.

۱۲ Generation Response

تولید پاسخ در سیستم فعلی شامل انتخاب بهترین منبع و ساخت پاسخ استخراجی است:

۱. بهترین candidate براساس score انتخاب می‌شود.
۲. متن پاسخ از answer_text، content یا chunk_text برداشته می‌شود.
۳. متن به جمله‌ها تقسیم می‌شود.
۴. جمله‌هایی که با توکن‌های سوال تطبیق بیشتری دارند انتخاب می‌شوند.

۵. پاسخ کوتاه و کنترل شده ساخته می شود.

در حالت پیشنهاد برای agent، این پاسخ در جدول ai_suggestions ذخیره می شود. در حالت پاسخ خودکار، widget اگر score کافی باشد یک پیام با sender_type = 'ai' در جدول messages ثبت می شود.

۱۳ Logic Fallback

منطق fallback یکی از نقاط مهم اعتمادپذیری سیستم است. AI فقط زمانی پاسخ مستقیم می دهد که شرایط زیر برقرار باشد:

- سایت فعال باشد و tenant فعال باشد.
- sites.ai_mode برابر off نباشد.
- ai_site_settings.assistant_enabled فعال باشد.
- برای auto_reply، auto_reply_enabled فعال باشد.
- برای auto_reply، پشتیبان انسانی در دو دقیقه اخیر آنلاین نباشد.
- برای auto_reply، تکراری برای همان پیام قبلا ثبت نشده باشد.
- confidence_score از آستانه min_auto_reply_score عبور کند.

اگر score پایین باشد، متن fallback ارسال یا پیشنهاد می شود و سوال در ai_unanswered_questions ذخیره می شود. این رفتار باعث می شود شکست پاسخ دهی به یک داده آموزشی عملی برای بهبود base knowledge تبدیل شود.

۱۴ Caching

در نسخه فعلی caching اختصاصی برای پاسخ های AI یا retrieval وجود ندارد. اما چند نوع جلوگیری/ cache از تکرار به صورت عملی دیده می شود:

- جلوگیری از پاسخ خودکار تکراری برای یک message_id از طریق بررسی ai_answer_logs.
- استفاده از hash برای URL و محتوا در ai_pages و ai_content_chunks.
- استفاده از ON DUPLICATE KEY UPDATE برای به روزرسانی صفحه خزیده شده به جای ایجاد رکورد تکراری.
- نگهداری دانش آماده شده در جداول AI، تا هر سوال نیازمند crawl یا پردازش مجدد HTML نباشد.

برای نسخه cache production، تنظیمات سایت، cache نتیجه جست و جو برای سوال های پرتکرار و queue برای های crawl سنگین پیشنهاد می شود.

۱۵ Selection Model

مدل فعال فعلی یک مدل rule-based/extractive داخلی است، نه یک مدل زبانی خارجی. انتخاب این مدل برای نسخه فعلی چند دلیل فنی دارد:

- پاسخ‌ها باید به دانش مشتری محدود و قابل توضیح باشند.
- هزینه پاسخ‌دهی برای SaaS چندمستاجری باید قابل کنترل باشد.
- محصول باید حتی بدون API خارجی در جریان اصلی کار کند.
- پشتیبان انسانی همچنان مسیر اصلی کنترل و اصلاح پاسخ است.

با این حال، متغیرهای محیطی OPENAI_API_KEY و OPENAI_MODEL نشان می‌دهند که معماری برای افزودن LLM آماده‌سازی اولیه دارد. مسیر تکامل مناسب این است که LLM فقط بعد از retrieval و با context محدود و guardrails citation/source استفاده شود، نه به‌عنوان پاسخ‌دهنده آزاد بدون منبع.

۱۶ Reasoning

Reasoning در این معماری به‌صورت symbolic و قابل توضیح انجام می‌شود. موتور به‌جای استدلال زبانی آزاد، چند تصمیم قابل ردیابی می‌گیرد:

- سوال کوتاه یا بدون توکن معنی‌دار رد می‌شود.
- intent و category با rule تشخیص داده می‌شود.
- منابع مرتبط با SQL و matching token پیدا می‌شوند.
- هر منبع با score عددی رتبه‌بندی می‌شود.
- اگر score به آستانه نرسد، fallback فعال می‌شود.
- همه تصمیم‌ها در ai_answer_logs و در صورت شکست در ai_unanswered_questions قابل پیگیری هستند.

این نوع reasoning برای کاربرد پشتیبانی مشتری مناسب است، چون خروجی آن قابل audit و قابل دفاع است.

۱۷ Tuning Fine

در نسخه فعلی fine-tuning مدل زبانی وجود ندارد. اما سیستم یک چرخه بهبود داده دارد که نقش عملی مشابه برای افزایش کیفیت پاسخ‌ها ایفا می‌کند:

۱. سوال visitor ثبت می‌شود.
۲. اگر AI پاسخ مطمئن پیدا نکند، سوال وارد صف unanswered می‌شود.
۳. admin customer سوال را بررسی می‌کند.
۴. admin می‌تواند آن را به base knowledge اضافه کند.

۵. پاسخ‌های بعدی با دانش جدید score بهتری می‌گیرند.

اگر در آینده LLM یا model embedding اضافه شود، داده‌های `ai_unanswered_questions`، `ai_answer_logs`، تصمیم‌ها `agent` درباره `ai_suggestions` و دانش تایید شده می‌توانند dataset مناسبی برای `prompt evaluation` یا `improvement` یا `fine-tuning` باشند.

۱۸ Evaluation

ارزیابی AI باید هم از نظر فنی و هم از نظر تجربه پشتیبانی انجام شود. معیارهای پیشنهادی:

معیار	روش اندازه‌گیری
Answer Rate	نسبت سوال‌هایی که score آن‌ها از آستانه suggestion یا auto reply عبور می‌کند.
Fallback Rate	نسبت سوال‌هایی که به fallback یا answer no منتهی می‌شوند.
Accepted Suggestion Rate	نسبت پیشنهادهایی که agent آن‌ها را، edited accepted، used می‌کند.
Unanswered Resolution Rate	نسبت سوال‌های بی‌پاسخ که توسط admin بررسی و به دانش اضافه می‌شوند.
Source Quality	بررسی اینکه پاسخ از دانش دستی، question generated یا chunk خام آمده است و کدام منبع موفق‌تر است.
Latency	زمان اجرای <code>ai_find_best_answer()</code> و <code>ai-suggestion-generate.php</code> و <code>ai-reply.php</code> endpoint‌ها.
Coverage by Intent	تعداد و کیفیت پاسخ‌ها در <code>intent</code> هایی مانند <code>contact</code> ، <code>pricing</code> ، <code>shipping</code> و <code>appointment</code> .
Safety	بررسی پاسخ‌های fallback، جلوگیری از پاسخ بدون منبع و جلوگیری از پاسخ مستقیم هنگام آنلاین بودن پشتیبان.

برای demo و ارزیابی پارک علم و فناوری، پیشنهاد می‌شود چند سناریوی واقعی از سایت نمونه آماده شود: سوال قیمت، سوال تماس، سوال خدمات، سوال نامرتبط و سوالی که باید fallback شود. سپس لاگ‌ها، `sources score` و مسیر تبدیل `unanswered` به `knowledge` نمایش داده شود.

۱۹ کنترل‌های امنیتی و دسترسی در AI

AI در این پروژه خارج از مدل امنیتی اصلی SaaS عمل نمی‌کند؛ بلکه به `tenant`، `role` site و `plan` محدود است.

- `admin customer` فقط سایت‌های `tenant` خودش را برای `crawl` و تنظیمات AI مدیریت می‌کند.
- `agent` و `admin customer` برای پیشنهاد AI باید به `conversation` و `site` دسترسی داشته باشند.

- endpoint پیشنهاد AI قابلیت plan با کلید ai_suggestions_enabled را بررسی می‌کند.
 - پاسخ خودکار widget با site_key، conversation، visitor، tenant message، validation origin status، site tus، و limit rate کنترل می‌شود.
 - source crawl باید به دامنه همان سایت تعلق داشته باشد و از بیرون دامنه مشتری پذیرفته نمی‌شود.
- نکته قابل بهبود برای production این است که در های fetch، crawler، SSL verification در کد فعلی غیرفعال است. برای محیط عملیاتی باید این رفتار بازبینی و ایمن‌تر شود.

۲۰ محدودیت‌های فعلی و مسیر توسعه

محدودیت	مسیر توسعه پیشنهادی
نبود embedding	افزودن embedding برای ai_content_chunks و ai_generated_questions و استفاده از search vector در کنار scoring فعلی.
نبود database vector	اتصال به pgvector، Qdrant یا Milvus یا استفاده از قابلیت برداری دیتابیس انتخابی در آینده.
Context محدود به آخرین پیام	افزودن resolver context برای سوال‌های پیگیری و خلاصه مکالمه.
Crawler همزمان در request	انتقال های crawl بزرگ به queue و worker با وضعیت queued/running/completed.
extraction Entity مستقل ندارد	استخراج ساختاریافته قیمت، شماره تماس، زمان، آدرس، نام محصول و خدمت.
Evaluation خودکار محدود است	ساخت benchmark داخلی از سوال/پاسخ‌های تاییدشده و محاسبه recall precision و latency.
LLM در جریان فعال استفاده نمی‌شود	افزودن LLM اختیاری بعد از retrieval با context محدود، guardrail و ثبت source.

۲۱ جمع‌بندی

معماری هوش مصنوعی AI Chat SaaS در نسخه فعلی یک معماری دانش‌محور، قابل کنترل و قابل توضیح است. این معماری از crawler، دانش دستی، قطعه‌بندی محتوا، generated extraction، term، ranking retrieval، detection، intent questions، پاسخ‌ها یا به agent پیشنهاد می‌شوند یا در صورت آفلاین بودن پشتیبانی و عبور از آستانه، به صورت خودکار در widget ارسال می‌شوند.

مزیت اصلی این طراحی برای محصول SaaS پشتیبانی مشتری این است که AI نقش کمک‌کننده دارد، نه جایگزین کامل انسان. agent می‌تواند پاسخ را ببیند، ویرایش کند یا رد کند؛ admin customer می‌تواند سوال‌های بی‌پاسخ را به دانش اضافه کند؛ و همه تلاش‌های پاسخ‌دهی در لاگ‌ها قابل بررسی هستند. بنابراین سیستم علاوه بر کاربرد عملی در demo، یک هسته فنی قابل توسعه برای RAG برداری، LLM

کنترل‌شده، ارزیابی خودکار و بهبود تدریجی دانش دارد.